TotEm: Carbon-Aware Kubernetes Scheduling with Embodied Emissions

Nasir Asadov, Vlad C. Coroamă, Martin Roch-Neirey, Marco Zambianco, Silvio Cretti, and Matthias Finkbeiner

Abstract—Data centers account for over 1% of global electricity use, yet today's container orchestrators optimize for performance and availability while ignoring carbon emissions that vary across time and location. At the same time, manufacturing ("embodied") emissions are a first-order contributor that existing schedulers do not model. We ask whether container scheduling can reduce total emissions—operational and embodied—without sacrificing feasibility or incurring prohibitive runtime, and how closely a practical scheduler can approach a MILP benchmark. To this end, we formulate a carbon-aware placement problem that integrates operational and embodied emissions under resource and timing constraints; design TotEm (Total Emissions scheduler), a fast heuristic that minimizes a comprehensive emissions objective; and implement a global mixed-integer linear program (MILP) as a strong benchmark, alongside a carbon-agnostic Kubernetes-like baseline. All methods are evaluated on identical workloads, horizons, carbon forecasts, and power/embodied models. Across scales, TotEm substantially reduces emissions while preserving high placement success and practical runtime. At a representative moderate-load point (≈ 28% implied average CPU utilization over 24 hours), average per-pod emissions fall from 36.6 to 24.7 g CO_2e (32.5%) relative to the carbon-agnostic baseline, while the MILP benchmark achieves 14.1 g. Runtime grows smoothly with pods and nodes and remains in single-digit seconds at our largest settings. These results establish carbon-aware container orchestration as a practical path to lower total emissions and provide a reproducible evaluation protocol for comparing heuristic and MILP approaches in sustainable cloud computing.

Index Terms—Carbon-aware computing, container orchestration, Kubernetes, sustainable computing, green cloud computing, optimization algorithms, mixed integer linear programming, global optimization.

1 Introduction

Cloud and edge data centers consume vast amounts of electricity [1], [2], and their carbon footprint continues to grow with the proliferation of containerized services and AI workloads. Today's container orchestration stacks, exemplified by Kubernetes, optimize primarily for resource utilization and service-level objectives while ignoring carbon-intensity signals that vary across regions and time [3]. At the same time, hardware manufacturing ("embodied" carbon) and operational emissions both contribute materially to the environmental impact of running distributed systems [4]. This gap between what orchestrators optimize and what matters environmentally motivates a principled, system-level approach to carbon-aware scheduling, as increasingly demonstrated in industry practice [5] and recent research [6].

However, making container orchestration carbon-aware is nontrivial. Practical schedulers must respect placement feasibility (CPU/RAM), temporal constraints (earliest start and deadlines), and dynamic demand patterns, all while coping with carbon signals that change over time and across regions. Incorporating both operational and embodied carbon into placement decisions further complicates the objective, and any approach must remain computationally tractable at cluster scale. This raises the central research question: can we reduce total carbon emissions through orchestration decisions without sacrificing feasibility or incurring prohibitive precomputation costs?

To this end, we present TotEm, a carbon-aware orchestration framework that explores a fast heuristic scheduler that greedily minimizes a combined operational+embodied carbon objective under resource and timing constraints. Complementarily, we also explore a MILP formulation with a lexicographic objective (first maximize successful placements, then minimize emissions) that we solve under a fixed time limit as a MILP benchmark (a carbon-emissions lower bound in our experiments); and a carbon-agnostic baseline that mirrors Kubernetes' resource-only scoring (a carbon-emissions upper bound) for our TotEm scheduler [7]. We integrate carbon-intensity forecasts with a co-location–aware power and embodied-carbon model, enforce earliest-start and deadline constraints, and evaluate all approaches in an offline precomputation mode that simulates multi-timeslot scheduling. Our experimental design systematically varies pods and nodes to assess both effectiveness and time complexity, producing apples-to-apples comparisons across algorithms.

We implemented a prototype scheduler and evaluated it offline on controlled workloads across 12–24 hour horizons while varying pod and node scales; at a representative moderate-load point ($\approx$ 28% implied average CPU utilization over 24 hours), TotEm reduced average per-pod

N. Asadov, V. C. Coroamă, and M. Finkbeiner are with the Department of Environmental Technology, Technische Universität Berlin, Strasse des 17. Juni 135, 10623 Berlin, Germany (e-mail: nasir.asadov@tu-berlin.de; coroama@tu-berlin.de; matthias.finkbeiner@tu-berlin.de).
M. Roch-Neirey is with the Department of Computer Systems Networking and Telecommunications, Polytechnique Montréal, Montreal, Canada (e-mail: martin.roch-neirey@hefr.ch).
M. Zambianco and S. Cretti are with Fondazione Bruno Kessler, 38123 Trento, Italy (e-mail: zambianco@fbk.eu; cretti@fbk.eu).
Corresponding author: Nasir Asadov (e-mail: nasir.asadov@tu-berlin.de).
Manuscript received Month Day, Year; revised Month Day, Year.

emissions from 36.6 to 24.7 g CO_2e (-32.5%) relative to a carbon-agnostic baseline, while end-to-end precompute remained in single-digit seconds.

Our results show that carbon-aware scheduling can substantially reduce emissions while preserving high placement success and practical runtime. TotEm achieves favorable carbon–performance trade-offs and scales efficiently across realistic pod and node counts, while the MILP benchmark establishes a strong lower bound on achievable emissions (and thus an upper bound on reductions) at a higher computational cost. Beyond demonstrating feasibility, we provide a reusable evaluation methodology—covering emissions modeling, constraint handling, and time-complexity sweeps—that enables rigorous, reproducible comparisons of carbon-aware orchestration strategies.

The remainder of this paper is organized as follows: Section 2 reviews related work. Section 3 introduces our carbon-aware scheduler, including accounting models (Section 3.1), problem formulation (Section 3.2), the MILP benchmark (Section 3.3), and the TotEm algorithm (Section 3.4). Section 4 details the experimental setup. Section 5 presents results. Section 6 discusses implications and limitations. Section 7 concludes with sustainability implications and future work.

2 Related Work

Carbon-aware workload scheduling leverages variations in the carbon intensity of electricity across *time* and *location*. Early research typically focused on *either* shifting workloads in time (temporal shifting) or across geographic locations (spatial shifting), but not both. A recent survey [8] found that most studies prior to 2023 pursued disjunct strategies—either temporal or spatial—rather than integrated approaches. Roughly 65% of works examined only one axis (time or space), though by 2024–2025 a growing share began to consider both. Beyond when and where workloads run, a complementary line of work examines how to allocate embodied (manufacturing) emissions in evaluating and optimizing schedulers. Standards such as SCI [9] and recent studies propose time- and utilization-aware attribution; we review these approaches and their implications in Subsection 2.3. The subsequent analysis does not aim to be an exhaustive presentation of the related work. Instead, it shows — via chosen examples — how the research landscape evolved from the early, disjunct spatial or temporal scheduling towards algorithms considering the two and those considering embodied emissions as well. For the early works in particular only a few archetypical examples are shown; a thorough analysis is provided in [8].

2.1 Disjunct Temporal or Spatial Scheduling

Temporal scheduling aims to defer workloads to periods when electricity is cleaner. For example, a carbon-aware Kubernetes scheduler that postpones non-critical batch jobs to predicted low-carbon hours was implemented in [10]. Using historical data to forecast carbon intensity, about 1.3% emissions reduction was achieved compared to default scheduling. Similar findings are reported in [11], indicating that selectively delaying tasks can meaningfully shrink emissions, especially under high renewable penetration and workload slack.

Spatial scheduling, by contrast, exploits geographic diversity in grid carbon intensity. Kubernetes was extended to prioritize data centers with lower carbon costs during job placement in [12]. More recently, a geo-distributed service manager that routes requests to greener regions while minimizing user-perceived latency was developed in [13]. An approach that schedules serverless functions across cloud regions based on real-time carbon efficiency was proposed in [14]. A model for VM placement across geo-distributed data centers, optimizing for carbon, cost, and energy efficiency jointly, was introduced in [15]. Spatial shifting can yield large gains when regions differ significantly in grid mix, provided available capacity at the receiving end.

2.2 Combined Spatio-Temporal Scheduling

Modern spatio-temporal schedulers seek to jointly optimize both *when* and *where* to execute workloads. A Kubernetes-based system that schedules jobs at the right cluster and time using carbon forecasts and Quality-of-Service (QoS) constraints was proposed in [16], achieving a 33% carbon footprint reduction compared to QoS-only baselines. A dynamic multi-cloud scheduler that integrates live carbon intensity data to dispatch workloads across cloud regions and time windows was introduced in [17]. A framework that provisions and schedules workloads for carbon savings, though with less emphasis on real-time orchestration, was presented in [18].

A cautionary note is offered in [19]: many workloads show limited additional benefit from spatio-temporal scheduling beyond what single-axis strategies can achieve. Modeling across 123 regions found that simple heuristics often captured most gains, and joint shifting showed diminishing returns in carbon-stable grids or inflexible workloads.

Despite these caveats, the direction of research clearly favors holistic strategies. Kubernetes has emerged as a key enabling platform, supporting carbon-aware extensions that allow integration of forecasting, regional carbon signals, and multi-cluster placement logic [8], [16]. The fusion of container orchestration and carbon intelligence is enabling practical deployments of spatio-temporal scheduling at scale.

2.3 Embodied Emissions Allocation in Carbon-aware Scheduling

Embodied emissions (during raw material extraction, manufacturing, logistics, and end-of-life of the underlying hardware) are increasingly recognized as an important contributor to a service's overall emissions and thus important to cover in carbon-aware scheduling. Yet, they are often omitted or uniformly amortized over calendar time. Two broad allocation strategies appear in the literature: (i) lifetime amortization (time-based), and (ii) utilization-aware attribution that splits embodied carbon by actual time-in-use and resource share.

Standards and frameworks. The Software Carbon Intensity (SCI) specification formalizes embodied allocation with

$$M = TE \cdot TS \cdot RS,$$

where TE is total device embodied emissions from LCA, TS is the time share (e.g., reserved or in-use fraction of expected lifetime), and RS is the resource share (e.g., vCPU, memory) attributed to the software component [9]. This structure subsumes both simple time-based amortization ($RS = 1$) and finer-grained attribution when detailed utilization or reservations are available.

Recent research. Beyond uniform amortization, several works incorporate embodied carbon into scheduling and hardware-management decisions: (i) carbon depreciation models that allocate a larger share of embodied carbon to newly provisioned servers and recover idle-time impacts during active use are proposed in [20], showing that traditional accounting can inadvertently favor premature refresh; (ii) aging-aware CPU core management that extends lifetime and reduces embodied-attributed emissions in inference clusters with minimal QoS impact is demonstrated in [21]; (iii) joint embodied+operational objectives with heterogeneity and age in dispatch are formulated in [22], yielding lower total emissions under utilization-aware lifetime models; and (iv) for edge contexts, co-optimizing embodied and operational carbon under intermittent utilization and tight resource constraints is advocated by [23].

Implications for scheduling. (1) Pure calendar-based amortization is appropriate when only coarse information is available, but it obscures idle periods and can inflate per-workload embodied intensity. (2) With utilization traces (e.g., hourly CPU), allocating embodied emissions in proportion to time-in-use and resource share yields more faithful attribution and can shift policies toward reusing capacity, consolidating strategically, or delaying refresh. (3) Accounting for device age and heterogeneity reveals that newer hardware, despite higher performance-per-watt, carries substantial embodied carbon that must be amortized through sustained use before outperforming older devices on total emissions. (4) These effects are amplified at the edge, where intermittency and under-utilization are common.

In this paper, we adopt a refined, SCI-consistent allocation: we attribute embodied emissions using time-in-use and resource share derived from hourly CPU utilization.

3 Carbon-Aware Scheduling Strategy

As stated from the outset, the aim of our carbon-aware scheduler is to minimize the carbon footprint (FP) for a random collection of nodes and pods. The carbon FP consists of operation and embodied carbon and therefore first and foremost we describe a model that accounts for both in Section 3.1. We continue with problem formulation in Section 3.2. Since this problem is NP-hard and finding a global optimum is computationally intractable at scale, we pursue two complementary approaches: Section 3.3 introduces a MILP benchmark to approximate the theoretical lower bound, while Section 3.4 presents our TotEm scheduler designed to capture most of these benefits with practical runtime.

3.1 Carbon Accounting Models

As the carbon footprint needs to be assessed before it can be allocated, we first establish the accounting foundations: the actionable carbon-intensity signal (Section 3.1.1) is multiplied by the energy consumption of the workload. For this purpose we use a node power model that maps pod requests to energy (Section 3.1.2). We elaborate on the actual amortization and allocation of embodied emissions in (Section 3.1.3). These components compose the per-candidate emissions objective used in the problem formulation (Section 3.2).

3.1.1 Operational: Carbon Signals

We use carbon intensity (ACI) as the actionable signal that varies by region and hour. Forecasts are essential because our scheduler chooses not just where to run a pod, but also when, within its deadline window. With a forward view of the grid, it can push flexible jobs toward cleaner hours, steer away from expected spikes, and, when options exist, favor greener regions.

We therefore treat a time- and location-indexed forecast as a shared input to all algorithms. Both the MILP benchmark and TotEm use it to score candidate placements; TotEm also uses it to quickly rank hours and to break ties consistently across nodes. We align the forecast horizon with the scheduling horizon (12–24 h in our experiments). Forecasts are imperfect, but using the same input for all algorithms keeps comparisons fair. Section 4 details sources, coverage, and preprocessing.

3.1.2 Operational: Power Consumption of Workloads

Each node exposes a two-point power profile with an idle baseline P_{idle} and a maximum P_{max}. We model node power $P(U)$ as a piecewise linear function of aggregate CPU utilization $U \in [0, 1]$, where $P(0) = 0$ and $P(U) = P_{idle} + (P_{max} - P_{idle}) \cdot U$ for $U > 0$. This implies that the idle baseline is incurred only when at least one workload is active, and is zero otherwise.

For a pod i with CPU request c_i on a node j with total capacity C_j^{tot} (see Table 2 for notation), we attribute per-pod power per slot t using the same linear interpolation for the incremental component and allocate the idle baseline across concurrent workloads in proportion to CPU share:

$$p_{ijt} = (P_{max} - P_{idle}) \cdot \frac{c_i}{C_j^{tot}} + \theta_{ijt} \cdot P_{idle}, \tag{1}$$

where $\theta_{ijt} = c_i / \sum_{k \in \mathcal{A}_{jt}} c_k$ is the allocation share of pod i among the set $\mathcal{A}_{jt}$ of all pods active on node j at time t, ensuring $\sum_i \theta_{ijt} = 1$ for active slots.

3.1.3 Embodied: Emissions and Amortization

The embodied carbon calculations are based on lifecycle assessment (LCA) data from the Boavizta database [24], which aggregates manufacturer-reported environmental product declarations following ISO 14040/14044 standards.

The dataset encompasses over 200 hardware configurations from major manufacturers including Dell, Lenovo, and others, providing representative embodied carbon values and expected lifetimes across different hardware categories. We computed category-wise averages for the device types used in our experimental evaluation, as summarized in Table 1.

The core challenge in embodied carbon accounting is properly amortizing the upfront manufacturing emissions

TABLE 1
Category-wise average embodied emissions and lifetimes

Hardware Category	Embodied Emissions (kg CO_2e)	Lifetime (years)
IoT	27.47	5.20
Smartphone	52.73	3.03
Laptop	231.86	4.13
Server	1230.66	3.87

over the hardware's operational lifetime and allocating them to individual workloads. Consistent with SCI's time-and-resource attribution, we amortize per hour of lifetime and allocate by resource share:

$$m_{jt}^{emb} = \frac{\mathrm{EC}_j}{L_j \times 365 \times 24} \tag{2}$$

where EC_j represents the total embodied emissions for a hardware unit (in grams CO_2e), L_j is the expected operational lifetime in years, and m_{jt}^{emb} is the amortized embodied carbon per hour of device lifetime.

For workload allocation, embodied emissions are attributed proportionally to both the time scheduled and the resource share within each active slot:

$$m_i^{emb} = \sum_{k=0}^{d_i-1} m_{j(t+k)}^{emb} \cdot \theta_{ij(t+k)} \tag{3}$$

where $\theta_{ij(t+k)}$ is the allocation share for pod i on node j at slot t (proportional to its resource share, with $\sum_i \theta_{ij(t+k)} = 1$ for active slots). This allocation model reduces to uniform time-based amortization when detailed shares are unavailable (i.e., $\theta_{ij(t+k)} = 1$ for a single active workload), and ensures that longer-running and higher-share workloads bear proportionally larger embodied carbon.

The embodied carbon calculation is integrated directly into the scheduling objective function, where it combines with operational emissions to provide a comprehensive carbon footprint assessment for each placement decision. This integrated approach ensures that both emission components influence scheduling decisions simultaneously, rather than treating them as separate optimization phases. We quantify the effect on placement patterns in Section 5.1.

3.2 Problem Formulation

We formulate the carbon-aware scheduling problem as a Mixed Integer Linear Programming (MILP) optimization that finds optimal assignments of pods (computational tasks) to nodes (computing resources) and timeslots while minimizing total carbon emissions incorporating both operational and embodied components. The following subsections detail the problem formulation.

3.2.1 Sets, Parameters and Decision Variables

Sets. Let $\mathcal{P}$ denote the set of pods (indexed by i), $\mathcal{F}$ the set of nodes (indexed by j), and $\mathcal{T}$ the set of timeslots (indexed by t).

Parameters. Table 2 lists the notation and description of the inputs used in the formulation.

TABLE 2
Glossary of parameters

Parameter	Description
$D_i \in \mathbb{N}^+$	Deadline timeslot for pod i
$s_i \in \mathbb{N}^+$	Earliest timeslot for pod i
$d_i \in \mathbb{N}^+$	Duration of pod i (in timeslots)
$w_i \in \mathbb{N}^+$	Temporal slack for pod i
$c_i \in \mathbb{R}^+$	CPU request of pod i
$r_i \in \mathbb{R}^+$	RAM request of pod i
$C_j^{tot} \in \mathbb{R}^+$	Total CPU capacity of node j
$C_{jt} \in \mathbb{R}^+$	Available CPU of node j in timeslot t
$R_{jt} \in \mathbb{R}^+$	Available RAM of node j in timeslot t
$p_{ijt} \in \mathbb{R}^+$	Power consumption of pod i on node j at time t
$m_{ijt} \in \mathbb{R}^+$	Total carbon mass of pod i on node j at time t
$m_{ijt}^{op} \in \mathbb{R}^+$	Operational carbon of pod i on node j at time t
$m_{ijt}^{emb} \in \mathbb{R}^+$	Embodied carbon of pod i on node j at time t
$\mathrm{CI}_{jt} \in \mathbb{R}^+$	Carbon intensity of node j in timeslot t
$\mathrm{EC}_j \in \mathbb{R}^+$	Embodied carbon of node j
$L_j \in \mathbb{R}^+$	Expected lifetime of node j
$u_j \in \mathbb{R}^+$	Utilization factor of node j
$\theta_{ijt} \in [0, 1]$	Allocation share of pod i on node j at time t

Decision Variables. The binary decision variable is defined as:

$$x_{ijt} = \begin{cases} 1 & \text{if pod } i \text{ placed on node } j \text{ at time } t \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

Feasible Placements. Let $\mathcal{V}_i \subset \mathcal{F} \times \mathcal{T}$ be the set of valid node-timeslot pairs for pod i:

$$\begin{aligned} \mathcal{V}_i = \{(j,t) \in \mathcal{F} \times \mathcal{T} \mid & t \geq s_i,\ t + d_i \leq D_i, \\ & C_{j(t+k)} \geq c_i\ \forall k \in \{0, \ldots, d_i - 1\}, \\ & R_{j(t+k)} \geq r_i\ \forall k \in \{0, \ldots, d_i - 1\}\} \end{aligned} \tag{5}$$

where the deadline timeslot $D_i = s_i + d_i + w_i$ includes the pod's execution duration and temporal slack for scheduling flexibility.

3.2.2 Objective Function

The optimization objective minimizes total carbon emissions across all scheduled pods:

$$\text{minimize} \sum_{i \in \mathcal{P}} \sum_{(j,t) \in \mathcal{V}_i} m_{ijt} \cdot x_{ijt} \tag{6}$$

where m_{ijt} represents the total carbon emissions for executing pod i on node j starting at timeslot t.

3.2.3 Carbon Emissions Model

The carbon emissions m_{ijt} incorporate both operational and embodied components over the pod's execution duration d_i:

$$m_{ijt} = \sum_{k=0}^{d_i-1} \left(m_{ij(t+k)}^{op} + m_{ij(t+k)}^{emb} \right) \tag{7}$$

where operational emissions $m_{ij(t+k)}^{op} = p_{ij(t+k)} \cdot \mathrm{CI}_{j(t+k)}$ depend on per-slot power $p_{ij(t+k)}$ and carbon intensity $\mathrm{CI}_{j(t+k)}$, and embodied emissions per timeslot are $m_{ij(t+k)}^{emb} = \frac{\mathrm{EC}_j}{L_j} \cdot \theta_{ij(t+k)}$ based on total embodied carbon EC_j, lifetime L_j, and allocation share $\theta_{ij(t+k)}$ (proportional to CPU share; $\sum_i \theta_{ij(t+k)} = 1$ for active slots).

3.2.4 Constraints

Resource capacity.

$$\sum_{i\in\mathcal{P}}\sum_{k=0}^{d_i-1} c_i\, x_{ij,\,t-k} \le C_{jt}, \tag{8}$$

$$\sum_{i\in\mathcal{P}}\sum_{k=0}^{d_i-1} r_i\, x_{ij,\,t-k} \le R_{jt}, \tag{9}$$

for all $j \in \mathcal{F}$ and $t \in \mathcal{T}$. We take $x_{ij,t} = 0$ when $t \notin \mathcal{T}$ or $(j,t) \notin \mathcal{V}_i$.

Assignment (per pod i).

$$\sum_{(j,t)\in\mathcal{V}_i} x_{ijt} = 1, \tag{10}$$

with the relaxed variant given in Section 3.3.1.

Temporal feasibility is encoded by the feasible set $\mathcal{V}_i$ defined earlier (earliest start and deadline windows, plus per-slot capacity checks along the pod's duration).

3.3 Theoretical Solution: MILP Benchmark

Before Section 3.4 presents our proposed algorithm, the current section first discusses the theoretically ideal solution: the mixed-integer linear program (MILP) that, in principle, globally optimizes scheduling decisions for minimum carbon under our model. In practice, solving this NP-hard problem to proven optimality quickly becomes intractable at scale, so we rely on a MILP implementation (Section 4.3) as a strong, but not always globally optimal, benchmark. The role of this MILP benchmark is twofold: First, to approximate lower-bound achievable emissions under identical data and constraints. Secondly, it is used in Sections 5 and 6 as one of the two reference points for our algorithm.

3.3.1 Feasibility and Relaxations

When the full MILP becomes infeasible (e.g., oversubscribed capacities or incompatible windows), we solve a relaxed variant that guarantees feasibility while preserving lexicographic priorities: (1) maximize pod placement; then (2) minimize total carbon given equal placement counts. We introduce binary slack variables y_i indicating unplaced pods and set a penalty $\lambda \gg \max m_{ijt}$ to enforce the priority ordering:

$$\text{Minimize} \quad Z(\mathbf{x},\mathbf{y}) = \lambda \sum_{i\in\mathcal{P}} y_i \tag{11}$$

$$+ \sum_{i\in\mathcal{P}} \sum_{(j,t)\in\mathcal{V}_i} m_{ijt} \cdot x_{ijt} \tag{12}$$

subject to the softened assignment constraint

$$\sum_{(j,t)\in\mathcal{V}_i} x_{ijt} + y_i = 1 \quad \forall i \in \mathcal{P}. \tag{13}$$

This hierarchy ensures responsiveness in intractable instances and yields upper bounds for heuristic comparisons.

3.3.2 Computational Complexity

The MILP contains $O(|\mathcal{P}|\,|\mathcal{F}|\,|\mathcal{T}|)$ binary variables over feasible pairs and is NP-hard with worst-case exponential time. Practical solvers (e.g., CBC or CP-SAT) employ branch-and-bound with cutting planes; performance is reasonable at moderate scales but degrades with increasing horizon and candidate sets. We therefore use solver time limits and the above relaxation to bound latency and to obtain MILP references at tractable sizes.

3.4 TotEm: Heuristic Carbon-Aware Scheduler

While the MILP formulation defines an optimal solution in principle, solving it to proven optimality becomes computationally intractable for large-scale deployments with hundreds of pods and nodes; in practice we rely on MILP runs as a high-quality reference. To address these challenges, we developed a spatio-temporal scheduling algorithm named TotEm. The principles of this algorithm were first presented in [8]; an improved—now implemented and benchmarked—version is presented in Algorithm 1 below.

The current paper introduces the further-developed version of this algorithm (Section 3.4.1). Section 4 details the experimental protocol, while Section 5 compares TotEm against both the carbon-agnostic baseline and the MILP benchmark.

3.4.1 Algorithmic Framework

The core algorithm maintains a priority queue of pending pods ordered by deadline urgency and evaluates all feasible (node, timeslot) combinations, scoring each candidate by e_{ijt} as defined in Section 3.2.3.

Constraint handling. A candidate (j,t) is feasible only if per-slot CPU and RAM capacities hold across the pod's duration and t respects deadline windows, as defined in Section 3.2.4.

Candidate ordering. Pods are processed by smallest scheduling window (deadline minus duration), then by larger CPU, RAM, and duration (stable sort). For each pod, we prioritize timeslots with lower forecasted carbon intensity. The scheduler iterates through these pre-ordered timeslots and, for each, enumerates all candidate nodes to find the best feasible placement.

Tie-breaking and determinism. Primary score is total emissions m_{ijt}. On ties (within a tiny ϵ), we prefer tighter packing using a normalized slack metric over CPU and RAM across the pod's duration (smaller is better). We set $\epsilon = 10^{-9}$ to avoid floating-point artifacts without affecting ranking. Determinism follows from the fixed ordering of pods, timeslots, and nodes.

Search strategy. We enumerate all feasible (node, timeslot) pairs for the pod (no node sampling or early-stop); the timeslot pre-ordering biases search toward lower-carbon hours.

Implementation notes. We track per-slot residual capacity in dictionaries indexed by node and timeslot, and use stable sorts for deterministic processing. Per-iteration work remains $O(|\mathcal{F}||\mathcal{T}|)$ per pod (see Section 3.3.2).

Unschedulable case. If no feasible (j, t) exists for a pod (e.g., due to capacity or window constraints), the pod remains pending and is retried in the next scheduling round; deadlines/windows are re-evaluated each pass. No implicit relaxation or forced placement occurs in TotEm.

Algorithm 1 TotEm Scheduler

Input: Pods $\mathcal{P}$, nodes $\mathcal{F}$, timeslots $\mathcal{T}$;
pod parameters $\{(c_i, r_i, d_i, s_i, D_i)\}_{i\in\mathcal{P}}$;
capacities $\{(C_{jt}, R_{jt})\}_{j\in\mathcal{F}, t\in\mathcal{T}}$;
carbon intensity $\{\mathrm{CI}_{j,t}\}_{j,t}$; power p_{ij}; embodied (EC_j, L_j)
Output: Schedule $S \subseteq \mathcal{P} \times \mathcal{F} \times \mathcal{T}$

Initialize residuals $\tilde{C}_{jt} \leftarrow C_{jt}$, $\tilde{R}_{jt} \leftarrow R_{jt}$ for all j, t; $S \leftarrow \emptyset$
Compute $w_i \leftarrow D_i - d_i$; sort $\mathcal{P}$ by $(w_i \uparrow,\ c_i \downarrow,\ r_i \downarrow,\ d_i \downarrow)$
for each $i \in \mathcal{P}$ **do**
  $m^\star \leftarrow \infty$, $(j^\star, t^\star) \leftarrow (\varnothing, \varnothing)$, $\pi^\star \leftarrow \infty$
  for each $t \in \mathcal{T}$ with $t \geq s_i$ and $t + d_i \leq D_i$ **do**
    for each $j \in \mathcal{F}$ **do**
      if $\tilde{C}_{j,t+\tau} \geq c_i$ and $\tilde{R}_{j,t+\tau} \geq r_i$ for all $\tau \in \{0, \ldots, d_i - 1\}$ **then**
        $\widehat{m}_{ijt} \leftarrow \sum_{\tau=0}^{d_i-1} \left(m^{op}_{ij,\,t+\tau} + m^{emb}_{ij,\,t+\tau}\right)$
        $\pi_{ijt} \leftarrow \sum_{\tau=0}^{d_i-1} \left(\frac{\tilde{C}_{j,t+\tau}-c_i}{C_{j,t+\tau}} + \frac{\tilde{R}_{j,t+\tau}-r_i}{R_{j,t+\tau}}\right)$
        if $\widehat{m}_{ijt} < m^\star - \epsilon$ **or** $(|\widehat{m}_{ijt} - m^\star| \leq \epsilon$ **and** $\pi_{ijt} < \pi^\star)$ **then**
          $m^\star \leftarrow \widehat{m}_{ijt}$, $(j^\star, t^\star) \leftarrow (j, t)$, $\pi^\star \leftarrow \pi_{ijt}$
        end if
      end if
    end for
  end for
  if $(j^\star, t^\star) \neq (\varnothing, \varnothing)$ **then**
    $S \leftarrow S \cup \{(i, j^\star, t^\star)\}$
    for $\tau = 0$ **to** $d_i - 1$ **do**
      $\tilde{C}_{j^\star,t^\star+\tau} \leftarrow \tilde{C}_{j^\star,t^\star+\tau} - c_i$; $\tilde{R}_{j^\star,t^\star+\tau} \leftarrow \tilde{R}_{j^\star,t^\star+\tau} - r_i$
    end for
  end if
end for
return S

3.4.2 Computational Complexity

TotEm algorithm has time complexity $O(|\mathcal{P}| \times |\mathcal{F}| \times |\mathcal{T}|)$ for each scheduling iteration, where $|\mathcal{P}|$ is the number of pending pods, $|\mathcal{F}|$ is the number of nodes, and $|\mathcal{T}|$ is the number of timeslots. This polynomial complexity represents a significant improvement over the MILP formulation.

4 Experimental Setup

To ensure clarity and reproducibility, we document the evaluation setup in the same order used to present results. Section 4.1 summarizes the system architecture (Fig. 1) to show where each scheduler runs. Section 4.2 details the testbed configuration and workload characteristics. Section 4.3 introduces the benchmarking baselines—a MILP benchmark and a carbon-agnostic scheduler. Section 4.4 outlines the evaluation methodology, including metrics, fairness protocols, and runtime measurement procedures.

4.1 System Architecture Overview

Our carbon-aware orchestration system is implemented as a distributed architecture that integrates with Kubernetes through a custom gRPC-based interface [25]. The system decouples carbon-aware scheduling logic from the underlying orchestration platform, enabling algorithmic experimentation.

The architecture in Figure 1 comprises four layers: (i) a client-facing ingress, (ii) a pluggable algorithm engine with three schedulers, (iii) internal data services, and (iv) external data sources. This separation (blue = core components/algorithms; green = internal data; orange = external sources) decouples scheduling logic from data acquisition and persistence.

Ingress and dispatch. A Go client running in Kubernetes calls a gRPC placement service that implements a stable service interface definition. The service forwards scheduling requests (cluster snapshot and workloads) to the algorithm engine, which dispatches to one of three schedulers: a carbon-agnostic baseline, TotEm (a real-time carbon-aware heuristic), or a MILP scheduler used for offline benchmarking.

Algorithms and data dependencies. TotEm and MILP benchmark consume carbon data (time- and location-varying forecasts) and hardware metadata (power curves, embodied carbon, lifetimes). The baseline interacts only with state storage and ignores carbon inputs. All algorithms read/write state storage to persist placements and metrics for reproducibility and experiment continuity.

Internal data services. Carbon data maintains rolling 24–48 hour forecasts aligned to regions/nodes. Hardware metadata stores per-node power/embodied parameters sourced from Kubernetes node labels and Boavizta LCA values. State storage persists scheduling state and experiment logs.

External sources. Electricity Maps [26] provides carbon-intensity forecasts that feed carbon data; Kubernetes node information supplies hardware attributes to hardware metadata; Boavizta DB supplies embodied-carbon and lifetime parameters.

Availability. We release the full implementation as open-source software [27] and provide a 'kubectl' plugin for cluster operators [28].

4.2 Testbed and Workloads

We evaluate all algorithms on a shared synthetic testbed with consistent infrastructure, workloads, and carbon forecasts:

Infrastructure and Nodes. Nodes are described by hardware metadata including region, embodied carbon and lifetime, and power profiles. Regions cover European grids with heterogeneous carbon intensity.

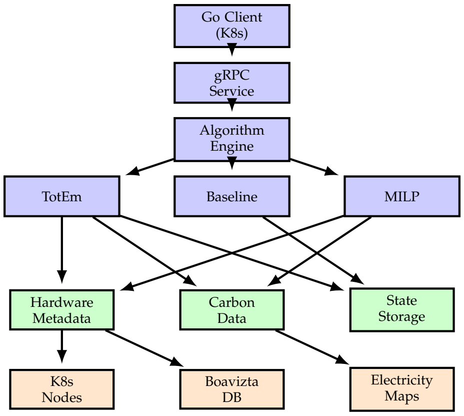

Fig. 1. Carbon-aware orchestration system architecture with component interactions and data flow. Color Key: Core components and algorithms Internal data services External data sources

Carbon Forecasts. Hourly average carbon-intensity (ACI) forecasts are loaded from a consolidated JSON (aligned across algorithms). Forecast horizons span 12–24 hours, enabling temporal shifting while respecting earliest/deadline constraints.

Scheduling Horizon. All methods operate over identical timeslot horizons. For fairness, feasibility checks require capacity across the entire execution duration of a pod for any candidate start slot.

Validation. A batch validator checks deadline, and capacity constraints on produced placements for all algorithms; current experiments pass with zero violations.

Workload Characteristics. Workloads are synthetically generated to simulate a heterogeneous cluster environment. Pods are assigned CPU and RAM requests sampled from a set of standard sizes (CPU: 0.1–2.0 cores; RAM: 128 MiB–2 GiB). Durations are selected from $\{1, 3, 6\}$ hours, and deadlines are defined as $D_i = s_i + d_i + w_i$, where the slack $w_i \in \{1, 3, 6\}$ hours. This structure creates bounded scheduling windows with varying flexibility. For controlled scaling experiments, we use an ”exact total” generation strategy that distributes a fixed number of pods (ranging from 20 to 400) across the scheduling horizon to precisely control workload density.

4.3 Benchmarking Baselines

The performance of the proposed algorithm is compared against two alternatives that together span the possible performance range of carbon orchestration: A plain carbon-agnostic scheduler represents the default vanilla choice. At the other end, a MILP scheduler (with oracle capabilities) approximates the best possible solution under our model.

MILP oracle. We instantiate the two-phase MILP benchmark as an oracle with full knowledge of all workloads arriving over the entire horizon. We use the following solver setup: OR-Tools CP-SAT by default (per-phase time limit 900 s, 8 threads), with PuLP fallbacks for HiGHS/Gurobi/CPLEX/CBC using the same time limit and a 5% relative-gap tolerance. Phase 2 uses binary activation variables to charge idle+embodied once per active (node, timeslot) with standard linking; we warm-start from Phase 1 where supported and, on timeout, run a dynamic-only fallback to return a usable bound. We log solver status, iteration counts (when available), and wall time. Because of this time limit (and, for MILP backends, the relative-gap stopping criterion), reported MILP emissions at larger scales should be interpreted as upper bounds on the true global optimum rather than certified optima.

Carbon-agnostic baseline. For a straightforward reference, we mirror Kubernetes’ *filter* → *score* → *select* path: (i) filter nodes that satisfy CPU/RAM across the pod’s duration at a candidate start, (ii) score feasible nodes with a LeastAllocated-like metric (equal CPU/RAM weights), and (iii) pick the best score at the earliest feasible start. It enforces earliest/deadline windows and ignores carbon in scoring. To keep behavior close to upstream, we enable node sampling and early feasible-stop so runtime and choices look like vanilla. Configuration: `percentage_of_nodes_to_score`=50%, `min_nodes_to_score`=50, `max_nodes_to_score`=1000, early stop after $k = 16$ feasible candidates. Feasibility uses a sliding-window check across the pod’s duration on each node. While the baseline ignores carbon during scheduling, we calculate the resulting total emissions offline using the same shared accounting model to ensure a fair comparison.

4.4 Evaluation Methodology

Metrics. We report:

- **Total emissions**: operational + amortized embodied (kg CO_2e), per run and per algorithm
- **Emissions per pod**: total divided by placed pods
- **Success rate**: fraction of pods placed (feasibility under constraints)
- **Runtime**: median/mean per-placement execution time and end-to-end precompute time

Protocol and Fairness. To quantify carbon reductions, we evaluate three schedulers—*carbon-agnostic* (baseline), *TotEm* (carbon-aware), and *MILP*—over controlled workload scales (20–200 pods) and time horizons. All algorithms operate under the same constraints (earliest timeslot, CPU/Memory, node capacity/affinity) and the same emissions model (operational + amortized embodied carbon; identical regional carbon-intensity forecasts and node power profiles). We fix seeds where stochasticity is used and report confidence intervals when averaging over three independent runs. Finally, we assess the robustness of our results through sensitivity analyses on forecast error and embodied carbon parameters.

Apples-to-apples comparison is non-trivial because algorithms can schedule different subsets of pods under the same load, confounding emissions with feasibility. We therefore present the coverage-inclusive view: emissions computed over all pods each algorithm successfully schedules, reflecting end-to-end impact (quality and coverage).

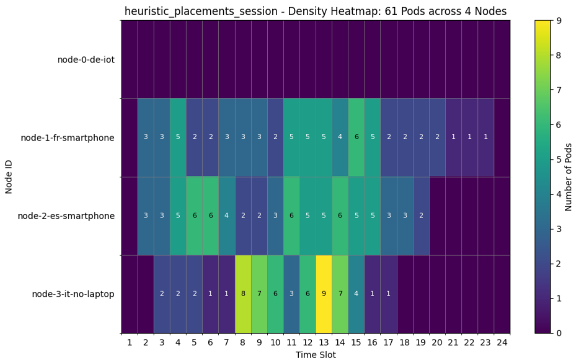

Fig. 2. Operational-only scheduling. Optimizing solely for electricity carbon intensity concentrates work on a subset of nodes, which can underutilize other hardware and inflate per-workload embodied attribution.

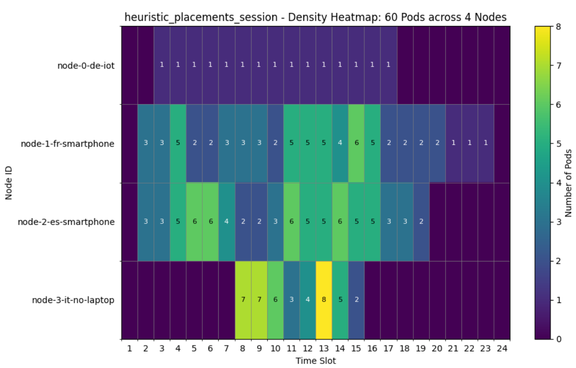

Fig. 3. Operational+embodied scheduling. Incorporating amortized embodied costs yields more balanced placements, improving lifetime utilization and total carbon efficiency.

Runtime Measurement. We report median precompute runtime in offline mode with a 12-slot horizon. For node-scaling overlays, we fix pods at 200 and vary node count; for pod-scaling overlays, we fix nodes at 16 and vary pod count. Curves overlay multiple schedulers; error bars (when present) denote $\pm 1\sigma$ over three replicates.

5 Results

We report how embodied emissions change placement behavior (Section 5.1), quantify emissions reductions and success rates (Section 5.2 and Fig. 6), and analyze runtime scaling (Section 5.4). Finally, we assess robustness to embodied parameters and forecast errors (Sections 5.5 and 5.6).

5.1 Impact of Embodied Emissions on Scheduling

Including embodied emissions fundamentally changes placement behavior: it shifts from clustering on low-CI nodes to more distributed utilization that amortizes fixed manufacturing emissions while still respecting operational carbon signals.

Taken together, Figures 2 and 3 show that accounting for embodied emissions alters the optimization landscape. The operational-only strategy prefers a small set of low-CI nodes, whereas the comprehensive objective prefers spreading load to amortize fixed embodied costs. This trade-off can increase operational emissions slightly in some slots but reduces total (operational+embodied) emissions across the horizon.

The result aligns with our allocation model: embodied emissions are a fixed, time-amortized cost that incentivizes sustained, efficient hardware use. Ignoring them underestimates true impact and can create seemingly efficient but globally suboptimal utilization patterns.

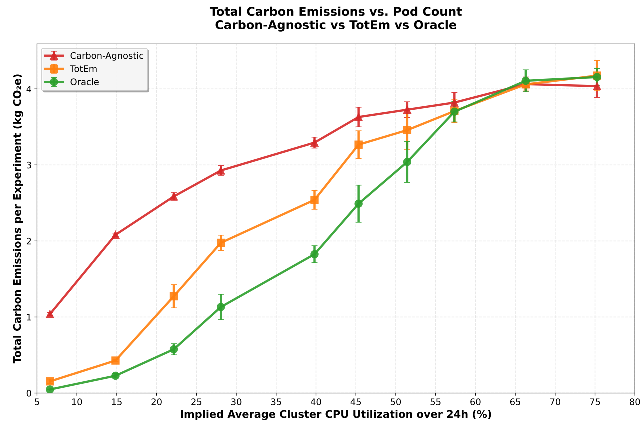

Fig. 4. Total carbon emissions vs. implied average cluster CPU utilization over 24 hours (coverage-inclusive). Emissions are computed using a shared model (operational + amortized embodied). Inclusive results reflect each algorithm's *actual* coverage at a given workload intensity.

5.2 Carbon Reduction

Figure 4 shows total emissions versus workload intensity, expressed as implied average cluster CPU utilization over a 24-hour horizon (coverage-inclusive). For each pod count, utilization is computed once from the workload generator as total requested CPU-core hours across all pods divided by the cluster's available CPU-core hours (30 cores $\times$ 24 hours); this mapping is shared across all algorithms. As load increases, the carbon-aware schedulers reduce emissions relative to the carbon-agnostic baseline; the MILP benchmark consistently provides the lowest emissions, and TotEm tracks it closely in most scales. Because inclusive totals aggregate whatever each algorithm could schedule, we jointly report success rate in Figure 6 (protocol in Section 4).

Across both views we use the same emissions accounting (operational + embodied). For completeness, per-pod or per-CPU-hour normalizations and operational-only sensitivity analyses can be included in the supplement; the qualitative ordering remains stable under these variants in our experiments.

Concrete findings. At this representative utilization ($\approx$ 28% implied average CPU utilization over 24 hours), per-pod emissions were: carbon-agnostic baseline 36.6 g CO_2e, TotEm 24.7 g ($\approx$-32.5%), and MILP 14.1 g ($\approx$-61.5%). Because the MILP is solved with a 900 s per-phase time limit (and, for MILP backends, a 5% relative-gap target), the 2.6 g CO_2e value should be interpreted as the best solution found under this budget—that is, an upper bound on the true global optimum—rather than a certified optimum. This gap nevertheless illustrates that TotEm captures most of the MILP benchmark's benefit while remaining deployable.

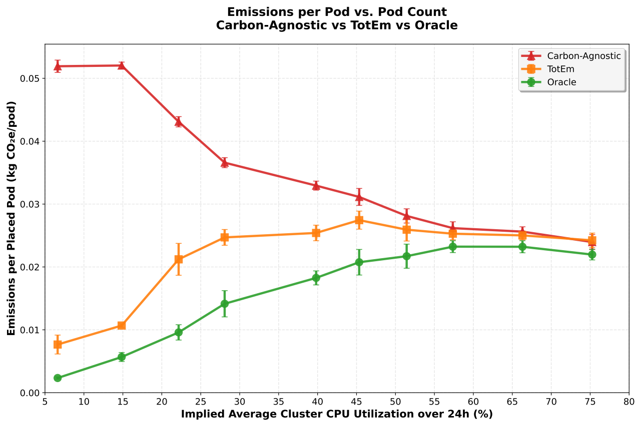

Fig. 5. Average emissions per placed pod (kg CO_2e/pod) vs. implied average cluster CPU utilization over 24 hours (coverage-inclusive). Carbon-aware methods remain nearly flat; vanilla starts high.

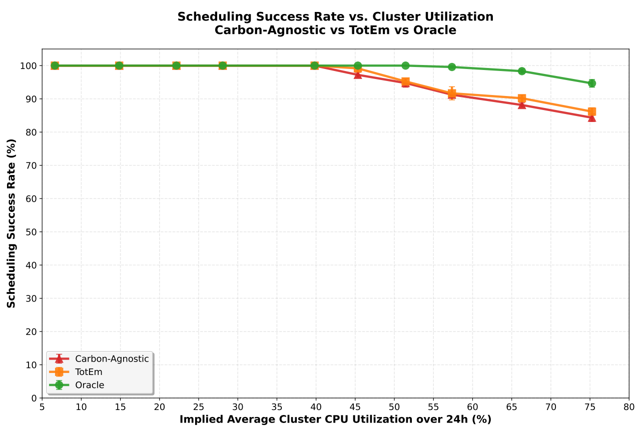

Fig. 6. Scheduling success rate vs. implied average cluster CPU utilization over 24 hours. Higher is better. Used jointly with inclusive emissions to disambiguate feasibility from carbon efficiency.

5.3 Placement Success Rate

Figure 6 shows scheduling success rate versus implied average cluster CPU utilization over 24 hours. At low load, all three methods are near 100%; as load increases, TotEm remains above the carbon-agnostic baseline and close to the MILP benchmark.

5.4 Runtime Cost: Time Complexity

Figures 7 and 8 summarise runtime. It grows roughly with the number of pods and remains compatible with online use at our largest scales. The MILP scheduler is far slower and sensitive to solver limits, so we treat it as a benchmarking tool rather than a production scheduler. Following the carbon-agnostic baseline optimizations described above, the baseline series shifts down substantially in both overlays.

Setup. We report median precompute runtime in offline mode with a 12-slot horizon. Figure 7 fixes pods to 200 and varies node count; Figure 8 fixes nodes to 32 and varies pod count. Curves overlay multiple schedulers; error bars (when present) denote $\pm 1\sigma$ over three replicates.

Results. Runtime grows smoothly with both nodes and pods. TotEm remains compatible with online use across tested

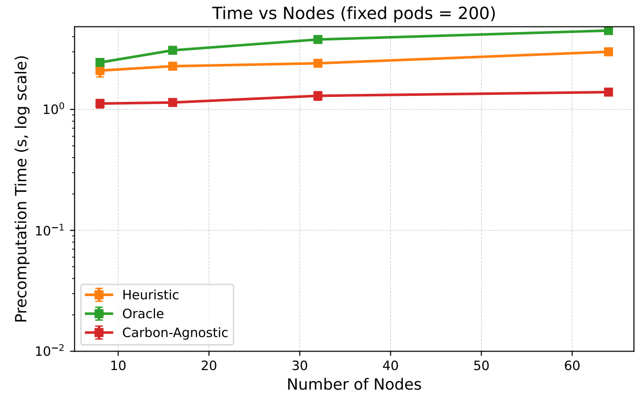

Fig. 7. Median precompute runtime versus node count at fixed 200 pods (overlay across algorithms; error bars where available denote $\pm 1\sigma$).

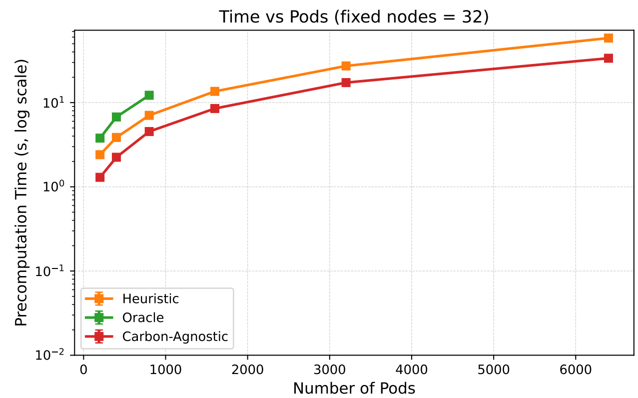

Fig. 8. Median precompute runtime versus pod count at fixed 32 nodes (overlay across algorithms; error bars where available denote $\pm 1\sigma$).

scales (single-digit seconds at the largest points), while the MILP grows much faster and is sensitive to solver limits. The carbon-agnostic baseline is generally the fastest (simplest scoring), with TotEm modestly above it but far below the MILP; this ordering holds in both fixed-pods and fixed-nodes views.

Fixed pods (varying nodes). With 200 pods held constant, both carbon-agnostic baseline and TotEm exhibit near-linear growth in precompute time as node count increases. The optimized baseline curve lies consistently below its earlier baseline, indicating lower constant factors; its slope remains modest due to node sampling and early termination once a small set of feasible candidates is found. By contrast, the MILP grows much more steeply with the number of nodes. Error bars are small across replicates, indicating stable timings and low run-to-run variance.

Fixed nodes (varying pods). With 32 nodes held constant, runtime grows approximately linearly with total pods for both carbon-agnostic baseline and TotEm, as expected from per-pod evaluation over a bounded timeslot window. The optimized baseline reduces both the intercept and the slope relative to the prior implementation, reflecting cheaper feasibility checks and less I/O in the hot path. The TotEm maintains a roughly constant factor overhead versus the baseline across scales—primarily from additional feasibility

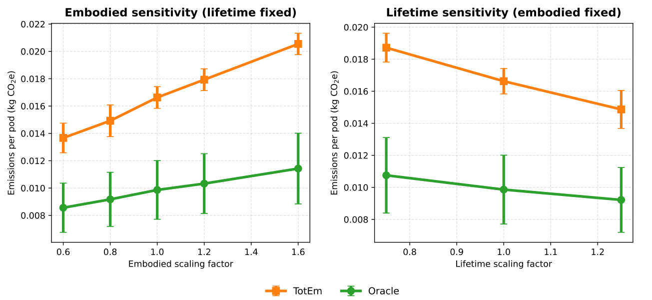

Fig. 9. Sensitivity of average per-pod emissions (kg CO_2e/pod) to embodied and lifetime scaling in the 80-pod scenario (mean with standard deviation across independent runs). Left: embodied scaling at fixed lifetime; right: lifetime scaling at fixed embodied emissions. Both TotEm and the MILP benchmark remain within a narrow band around the baseline.

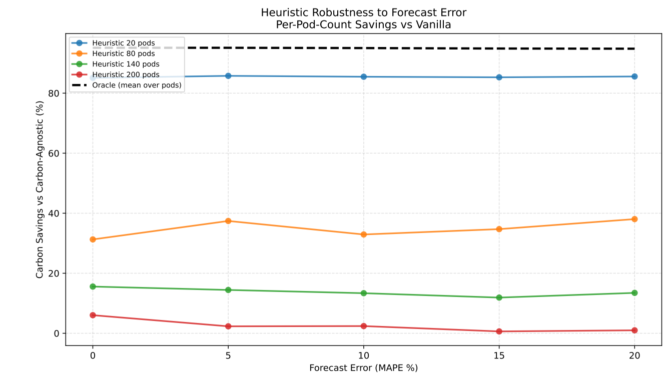

Fig. 10. Carbon savings relative to the carbon-agnostic baseline when schedulers consume noisy forecasts with various MAPE levels. Thin lines show TotEm savings for four representative pod counts; the dashed line shows the MILP bound averaged over pod counts. Decisions use the noisy traces; scoring uses the original Electricity Maps trace.

and scoring work—while remaining far below the MILP benchmark, which grows faster-than-linearly and exhibits higher variance.

Takeaways. (i) Empirical scaling matches the intended design: linear in pods at fixed nodes, and close to linear in nodes at fixed pods; (ii) kube-inspired pruning (node sampling and early feasible stop) keeps growth in check; (iii) MILP remains orders of magnitude slower and is best treated as a benchmark for bounds rather than a production scheduler.

5.5 Sensitivity to Embodied Assumptions

Embodied emissions and lifetimes in Table 1 are derived from Boavizta [24], but real deployments see variation between individual hardware configurations and management policies. To test that our conclusions do not hinge on exact values, we run a one-factor-at-a-time sensitivity study at the same representative utilization level. Guided by the Boavizta variability analysis, we apply uniform perturbations across all node classes: embodied emissions scaled by $\{0.60, 0.80, 1.00, 1.20, 1.60\}$ and lifetimes by $\{0.75, 1.0, 1.25\}$, averaged over independent seeds.

Figure 9 shows that these perturbations change average per-pod emissions by at most $\approx \pm 25\%$ for TotEm and $\approx \pm 17\%$ for the MILP benchmark across embodied scaling, and within $\approx \pm 15\%$ (TotEm) / $\approx \pm 10\%$ (MILP) across lifetime scaling—ranges consistent with the Boavizta-derived variability band. The qualitative picture remains intact: TotEm and the MILP benchmark stay close to each other, both remain well below the carbon-agnostic baseline, and the ordering observed in Section 5.2 is preserved. This suggests that our main findings about carbon-aware scheduling and the TotEm–MILP gap are robust to reasonable uncertainty in embodied-carbon and lifetime parameters.

5.6 Sensitivity to Forecast Error

Protocol. To test robustness we introduce controlled noise into the Electricity Maps trace and regenerate schedules using the same experimental setup as in Section 4. We inject multiplicative Gaussian noise with calibrated scale factors that reach target mean absolute percentage errors (MAPE) of 0, 5, 10, 15, and 20%; for each level we rerun the carbon-agnostic baseline, TotEm, and the MILP scheduler across pod counts from 20 to 200 and three random seeds. Each scheduler observes only the noisy trace while making decisions, but we evaluate the resulting placements against the untouched trace to approximate real-world execution with forecast error. We report carbon savings relative to the optimized carbon-agnostic baseline after resampling nodes' carbon curves and recomputing operational+embodied emissions.

Findings. Figure 10 shows that the TotEm's robustness depends strongly on system scale, while the MILP bound is remarkably stable. For small experiments (e.g., 20 pods), TotEm maintains large savings even under 20% MAPE, but as pod count increases its curve slopes downward and the spread between lines widens, indicating that forecast noise can substantially erode savings in dense regimes. In contrast, the MILP remains near $\approx 95\%$ savings for all noise levels and pod counts, confirming that much of the remaining gap is due to limited search rather than an inherent brittleness of carbon-aware placement. Because the carbon-agnostic baseline ignores forecasts entirely, its emissions are identical across the sweep, so all variation in the figure comes from how carbon-aware schedulers respond to imperfect predictions.

6 Discussion

We interpret the emissions patterns (why per-pod curves are flat and why the vanilla baseline starts high), discuss implications and limitations, and outline threats to validity, tying back to the methods in Section 4 and results in Section 5.

Revisiting our questions. Our results indicate that (i) feasibility remains high under carbon-aware placement with realistic slack, (ii) the TotEm approaches MILP's carbon outcome across scales while running orders of magnitude faster, and (iii) precompute latency remains compatible with nearline or periodic admission control, suggesting practical deployability.

6.1 Interpreting Emissions Patterns

Why per-pod curves are flat. With co-location–aware accounting, dynamic power for a node-hour is linear in aggregate utilization U, and per-pod attribution scales with CPU share. The pod-normalized dynamic term simplifies to CI $\times$ $k \times u_i$, independent of the number of co-located pods. Idle and embodied are paid *once* per active node-hour and allocated proportionally to CPU share, so a pod's share $\propto u_i/U$ *shrinks* as utilization grows. As a result, average emissions per pod stay approximately constant (or slightly decrease) even as the system fills and placement flexibility declines.

Why vanilla starts high. The vanilla scheduler's LeastAllocated-like scoring spreads pods to underutilized nodes, activating many node-hours early. This incurs idle and amortized embodied costs across more nodes and often places work in high-CI nodes/timeslots from the outset, leading to substantially higher per-pod emissions relative to carbon-aware schedulers that consolidate selectively and align with low-CI windows.

6.2 Implications and Limitations

How it scales. In practice, runtime grows with the number of pods and nodes that must be evaluated. Two things keep it fast: (i) realistic deadlines shrink the number of eligible start times per pod, and (ii) early feasibility checks discard many node–time candidates before detailed scoring. Memory use grows with tracked capacity per node and the number of pods under consideration.

Empirical behavior. Across our sweep (12-slot horizon; 8–64 nodes; 50–400 pods; three replicates), runtime increased smoothly with load and remained under eight seconds at the largest point, suitable for online or nearline admission control.

Implications for deployability. When response-time targets are tight, we can further cap per-pod evaluations to a small set of promising nodes (e.g., by carbon intensity), reducing constant factors while preserving outcomes.

When it slows down. Very loose deadlines, many nodes, or long durations increase candidates to check and can slow evaluation. Rich policy constraints also add per-candidate checks. The MILP, by contrast, can be much slower and is retained as a benchmarking tool rather than for production.

Further speedups. We can reduce overheads further by caching per-node energy/carbon curves, reusing capacity snapshots across similar pods, and evaluating batches of candidates together.

6.3 Threats to Validity

Internal validity. We control randomness via fixed seeds and validate constraints post hoc. Remaining risks include generator-induced artifacts; we mitigate by sweeping multiple pod counts and infrastructures.

Construct validity. We use ACI-based operational emissions and amortized embodied accounting. Alternative signals (e.g., marginal intensity, excess renewable) could change relative benefits.

External validity. Results derive from synthetic workloads and consolidated forecasts; real clusters may have additional constraints (affinity/anti-affinity, topology spread, storage). Our simulator architecture allows incrementally adding these.

Conclusion validity. We report multiple scales and pair emissions with success rate and runtime to avoid over-attributing gains to any single metric.

7 Conclusion and Future Work

This paper shows that container orchestration can reduce *total* emissions—operational and embodied—without sacrificing feasibility or incurring prohibitive runtime. Across controlled scales, TotEm consistently lowers emissions relative to a carbon-agnostic baseline while closely tracking the MILP benchmark. At a representative moderate-load point ($\approx$ 28% implied average CPU utilization over 24 hours), average per-pod emissions dropped from 36.6 to 24.7 g CO_2e (-32.5%) versus baseline, with the MILP reaching 14.1 g. Runtime grows smoothly with pods and nodes and remains within single-digit seconds at our largest settings, indicating practicality for precompute or nearline use.

For practitioners, these results suggest an actionable path: integrate forecasted carbon signals and amortized embodied costs into placement, and deploy a fast heuristic to capture most of the achievable benefit. Our evaluation protocol—shared signals, models, constraints, and coverage-inclusive reporting—provides a reproducible basis for comparing algorithms and for tuning trade-offs between feasibility, carbon efficiency, and latency.

Our study has limitations. We evaluate on synthetic workloads and consolidated forecasts; real clusters introduce additional constraints (e.g., affinity, topology spread, storage) and dynamic interference. We use ACI as the operational signal and amortized embodied accounting; alternative signals (e.g., marginal intensity, excess-renewable availability) and allocation choices may shift outcomes. Network and migration energy are not modeled, and our experiments run in an offline precomputation mode.

Future work will: (i) integrate additional environmental indicators (e.g., water footprint, land use etc.); and (ii) evaluate on production traces and in live clusters with online admission. Together, these steps move carbon-aware orchestration from controlled studies toward robust production deployments.

Acknowledgments

The authors would like to extend their gratitude to Electricity Maps for providing access to their carbon intensity data, which was invaluable for the development of this work.

References

[1] International Energy Agency, "Data centres and data transmission networks," https://www.iea.org/reports/data-centres-and-data-transmission-networks, 2024, iEA Analysis; electricity use, growth, and forecasts.
[2] E. Masanet, A. Shehabi, N. Lei, S. Smith, and J. Koomey, "Recalibrating global data center energy-use estimates," *Science*, vol. 367, no. 6481, pp. 984–986, 2020.

[3] Ember, "Global electricity review 2024," https://ember-climate.org/insights/research/global-electricity-review-2024/, 2024.

[4] Z. Cao, X. Zhou, X. Wu, Z. Zhu, T. Liu, J. Neng, and Y. Wen, "Data Center Sustainability: Revisits and Outlooks," *IEEE Transactions on Sustainable Computing*, vol. 9, no. 3, pp. 236–248, May 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10139829/

[5] A. Radovanovic, J. Dean *et al.*, "Carbon-intelligent computing at google," Google AI Blog, 2020. [Online]. Available: https://ai.googleblog.com/2020/05/carbon-intelligent-computing-at-google.html

[6] L. Gu, W. Zhang, Z. Wang, D. Zeng, and H. Jin, "Service Management and Energy Scheduling Toward Low-Carbon Edge Computing," *IEEE Transactions on Sustainable Computing*, vol. 8, no. 1, pp. 109–119, Jan. 2023. [Online]. Available: https://ieeexplore.ieee.org/document/9905948/

[7] Kubernetes Authors, "Scheduling framework," https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/, 2025.

[8] N. Asadov, V. C. Coroamă, M. Franzil, S. Galantino, and M. Finkbeiner, "Carbon-aware spatio-temporal workload shifting in edge–cloud environments: A review and novel algorithm," *Sustainability*, vol. 17, no. 14, 2025. [Online]. Available: https://www.mdpi.com/2071-1050/17/14/6433

[9] Green Software Foundation, "Software Carbon Intensity (SCI) Specification," 2024. [Online]. Available: https://sci.greensoftware.foundation/

[10] T. Piontek, K. Haghshenas, and M. Aiello, "Carbon emission-aware job scheduling for Kubernetes deployments," *The Journal of Supercomputing*, Jun. 2023. [Online]. Available: https://link.springer.com/10.1007/s11227-023-05506-7

[11] P. Wiesner, I. Behnke, D. Scheinert, K. Gontarska, and L. Thamsen, "Let's Wait Awhile: How Temporal Workload Shifting Can Reduce Carbon Emissions in the Cloud," in *Proceedings of the 22nd International Middleware Conference*, Dec. 2021, pp. 260–272, arXiv:2110.13234 [cs]. [Online]. Available: http://arxiv.org/abs/2110.13234

[12] A. James and D. Schien, "A Low Carbon Kubernetes Scheduler," 2019.

[13] A. Souza, S. Jasoria, B. Chakrabarty, A. Bridgwater, A. Lundberg, F. Skogh, A. Ali-Eldin, D. Irwin, and P. Shenoy, "CASPER: Carbon-Aware Scheduling and Provisioning for Distributed Web Services," in *Proceedings of the 14th International Green and Sustainable Computing Conference*, ser. IGSC '23. New York, NY, USA: Association for Computing Machinery, May 2024, pp. 67–73. [Online]. Available: https://dl.acm.org/doi/10.1145/3634769.3634812

[14] M. Chadha, T. Subramanian, E. Arima, M. Gerndt, M. Schulz, and O. Abboud, "GreenCourier: Carbon-Aware Scheduling for Serverless Functions," in *Proceedings of the 9th International Workshop on Serverless Computing*, ser. WoSC '23. New York, NY, USA: Association for Computing Machinery, Dec. 2023, pp. 18–23. [Online]. Available: https://doi.org/10.1145/3631295.3631396

[15] S. Rawas, A. Zekri, and A. El-Zaart, "LECC: Location, energy, carbon and cost-aware VM placement model in geo-distributed DCs," *Sustainable Computing: Informatics and Systems*, vol. 33, p. 100649, Jan. 2022. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2210537921001323

[16] T. Bahreini, A. N. Tantawi, and O. Tardieu, "Caspian: A Carbon-aware Workload Scheduler in Multi-Cluster Kubernetes Environments," in *2024 32nd International Conference on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*, Oct. 2024, pp. 1–8, iSSN: 2375-0227. [Online]. Available: https://ieeexplore.ieee.org/document/10786568/?arnumber=10786568

[17] B. M. Beena, P. C. Ranga, T. S. S. Manideep, S. Saragadam, and G. Karthik, "A Green Cloud-Based Framework for Energy-Efficient Task Scheduling Using Carbon Intensity Data for Heterogeneous Cloud Servers," *IEEE Access*, vol. 13, pp. 73 916–73 938, 2025. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/10971939

[18] W. A. Hanafy, L. Wu, D. Irwin, and P. Shenoy, "CarbonFlex: Enabling Carbon-aware Provisioning and Scheduling for Cloud Clusters," May 2025, arXiv:2505.18357 [cs]. [Online]. Available: http://arxiv.org/abs/2505.18357

[19] T. Sukprasert, A. Souza, N. Bashir, D. Irwin, and P. Shenoy, "On the Limitations of Carbon-Aware Temporal and Spatial Workload Shifting in the Cloud," in *Proceedings of the Nineteenth European Conference on Computer Systems*. Athens Greece: ACM, Apr. 2024, pp. 924–941. [Online]. Available: https://dl.acm.org/doi/10.1145/3627703.3650079

[20] S. Ji, Z. Yang, A. K. Jones, and P. Zhou, "Towards Datacenter Environmental Sustainability Using Carbon Depreciation Models," Mar. 2025, arXiv:2403.04976 [cs]. [Online]. Available: http://arxiv.org/abs/2403.04976

[21] T. B. Hewage, S. Ilager, M. R. Read, and R. Buyya, "Aging-aware CPU Core Management for Embodied Carbon Amortization in Cloud LLM Inference," Jan. 2025, arXiv:2501.15829 [cs]. [Online]. Available: http://arxiv.org/abs/2501.15829

[22] Y. Zhang, Y. Song, and S. Sahoo, "Carbon and Reliability-Aware Computing for Heterogeneous Data Centers," Apr. 2025, arXiv:2504.00518 [eess]. [Online]. Available: http://arxiv.org/abs/2504.00518

[23] A. M. Panteleaki, V. Paramanayakam, V. Pentsos, A. Karatzas, S. Tragoudas, and I. Anagnostopoulos, "A Vertical Approach to Designing and Managing Sustainable Heterogeneous Edge Data Centers," Jun. 2025, arXiv:2506.01712 [eess]. [Online]. Available: http://arxiv.org/abs/2506.01712

[24] "Boavizta/environmental-footprint-data," Jun. 2023, original-date: 2020-12-30T15:01:59Z. [Online]. Available: https://github.com/Boavizta/environmental-footprint-data

[25] S. Cretti, "fogatlas-k8s / algorithms · GitLab," Apr. 2024. [Online]. Available: https://gitlab.fbk.eu/fogatlas-k8s/algorithms

[26] Electricity Maps, "Live 24/7 co_2 emissions of electricity consumption," 2024. [Online]. Available: http://electricitymap.tmrow.co

[27] N. Asadov, "nasadov/carbon-aware-orchestrator," Oct. 2025, original-date: 2025-02-28T17:05:28Z. [Online]. Available: https://github.com/nasadov/carbon-aware-orchestrator

[28] S. Cretti and N. Asadov, "nasadov/kubectl-carbon," Sep. 2025, original-date: 2025-04-14T10:02:09Z. [Online]. Available: https://github.com/nasadov/kubectl-carbon